

Feasibility Analysis and Limitations of Tile-Wise Entropy Compression for SRAM-Constrained Edge Inference

Ahamed Nishan A
SARCS, CVEST
IIIT, Hyderabad
Hyderabad, India

Dr Priyesh Shukla
SARCS, CVEST
IIIT, Hyderabad
Hyderabad, India

Abstract—Deploying transformer-based neural networks on edge devices is severely constrained by the limited capacity and bandwidth of on-chip SRAM. While architectural projections suggest that combining quantization, sparsification, and entropy coding can drastically minimize memory footprints, the practical limits of applying general-purpose lossless compression to heavily quantized neural network weights require empirical validation rather than assumption.

This paper presents a hardware-software feasibility study of a memory-centric inference architecture combining SparseGPT pruning, 4-bit activation-aware weight quantization (AWQ), and lossless tile-wise Zstandard (Zstd) compression with SRAM-resident semantic dictionaries. Streaming hardware decompression and KV-cache compression datapaths were implemented and verified in RTL against independent golden reference models. Empirical measurement on the AWQ-INT4-quantized Qwen3-8B checkpoint reveals a critical entropy bottleneck: tile-wise Zstd compression contributes only a $1.09\text{--}1.11\times$ reduction beyond quantization alone, against a measured nibble entropy of 3.65 of a possible 4.0 bits/symbol. We further show, at per-tensor granularity, the specific mechanism by which quantization produces this outcome: AWQ’s per-group affine scaling relocates essentially all of the weight distribution’s exploitable structure into scale/zero-point side tensors that compress at $\approx 1.98\times$ but constitute under 1% of stored bytes, while the residual 4-bit codes – over 99% of the bytes – compress at only $\approx 1.08\times$. Quantization therefore does not merely fail to leave redundancy behind; it actively separates the compressible and incompressible parts of the representation and concentrates the former where compression cannot help. A direct measurement on the unquantized dense checkpoint, using identical methodology, confirms this is an effect of quantization rather than an inherent property of the weight data: Zstd achieves $1.254\times$ (a 20.3% size reduction) on FP16 weights against $1.107\times$ (9.7%) on AWQ-INT4 – roughly double the relative yield – even though AWQ-INT4’s own $3.63\times$ packing gain still leaves the combined AWQ+Zstd pipeline $\approx 3.2\times$ smaller in absolute terms than FP16+Zstd. Given this measured $\sim 10\%$ margin, together with the SRAM, control, and verification overhead required to realize it in hardware, we conclude that further investment in entropy-coding accelerators for post-quantization edge inference carries a high burden of proof, and that memory-reduction effort is better directed at sparsification and quantization design.

Index Terms—Edge AI, Large Language Models, Negative Results, Model Compression, AWQ, Zstandard, KV Cache, Hardware Accelerator, SRAM.

I. INTRODUCTION

The increasing size of transformer-based neural networks has created a fundamental memory bottleneck for edge inference. Storing and moving parameters remains expensive when the target device has limited SRAM or DRAM, and this problem scales dynamically during autoregressive inference due to the growing key-value (KV) cache.

Several techniques have emerged to reduce model memory requirements. SparseGPT demonstrated that large language models can be pruned in a single pass while retaining accuracy [2]. AWQ subsequently introduced activation-aware 4-bit weight-only quantization designed specifically for efficient LLM deployment [3]. Hardware acceleration of general-purpose compression has separately been investigated for large-scale systems, motivated by the processor and memory costs of software compression at scale [5]; it is natural to ask whether the same class of hardware is worth building for edge LLM weight storage specifically.

A recurring architectural proposal is to stack a further, lossless entropy-coding stage – such as Zstandard (Zstd) [4], [6] or Huffman coding [7] – on top of an already-quantized representation, on the assumption that some residual statistical redundancy remains to be extracted. This assumption is testable rather than self-evident: any lossless code, however implemented, is bounded below by the source’s entropy [8], and a well-designed quantizer’s entire purpose is to remove exactly the kind of statistical redundancy an entropy coder would otherwise exploit. Whether meaningful redundancy survives 4-bit activation-aware quantization in practice is an empirical question this paper answers directly.

This paper documents the design, RTL implementation, and evaluation of a unified model-compression hardware pipeline built to test this question, and reports a definitive negative result: applying 4-bit quantization consumes most of the statistical redundancy an entropy coder relies on, and – as we show at per-tensor granularity – does so through a specific, identifiable mechanism rather than merely as an incidental side effect. The contributions of this paper are:

- A hardware decompression datapath and a hardware KV-

cache write-side compressor, implemented in SystemVerilog RTL and verified against independent golden C models, including single-tile bit-exact entropy decode under two adversarial verification checks.

- A direct empirical measurement of the lossless compression headroom remaining in AWQ-INT4-quantized weights, at both whole-tensor and hardware-realistic tile granularity, with and without a trained dictionary.
- An identified, data-supported *mechanism* for this result: AWQ’s group-wise scale/zero-point factorization relocates compressible structure into a side channel representing under 1% of stored bytes, leaving the dominant 4-bit weight codes close to maximal entropy.
- A direct, measured comparison against unquantized FP16 weights under identical methodology, showing that quantization does not merely fail to leave redundancy behind but measurably suppresses it: Zstd recovers 20.3% of remaining size on FP16 against only 9.7% on AWQ-INT4, even though AWQ-INT4’s own $3.63\times$ packing gain leaves the combined pipeline substantially smaller overall.
- An explicit accounting of what remains unmeasured (principally KV-cache compressibility and standards-compliant Zstd throughput), so that the negative conclusion is properly scoped rather than overextended.

II. BACKGROUND AND MOTIVATION

A. Model Preparation Pipeline

The investigated compression pipeline begins from the dense FP16 representation, passing through pruning and quantization before reaching the lossless compression phase:

$$W_{\text{FP16}} \xrightarrow{\text{SparseGPT}} W_{\text{sparse}} \xrightarrow{\text{AWQ}} W_{\text{INT4}} \xrightarrow{\text{Tile}} \{T_i\} \xrightarrow{\text{Zstd}} \{C_i\} \quad (1)$$

SparseGPT calculates which parameters can be eliminated while minimizing reconstruction error [2]. AWQ then maps the resulting weights to a low-bit representation [3]. AWQ-INT4 quantization and, separately, SparseGPT (50%) pruning have each been applied to the reference Qwen3-8B checkpoint [1] and their storage ratios measured directly; combining the two stages on a single checkpoint – pruning followed by quantization of the resulting sparse weights – has not yet been performed. The two combined rows in Table I therefore remain literature-informed projections rather than measurements, though the fact that both individual-stage measurements landed within rounding of their originally projected values (Section V-A) is evidence in favor of, without confirming, the combined projection.

B. AWQ 4-Bit Quantization

AWQ, in its group-wise form, partitions each weight matrix along its input-channel dimension into contiguous groups of size G (commonly $G = 128$), and fits an independent affine quantizer (s_g, z_g) per group g :

$$q_{g,i} = \text{clip}\left(\text{round}\left(\frac{w_{g,i}}{s_g}\right) + z_g, 0, 15\right), \quad (2)$$

with reconstruction

$$\hat{w}_{g,i} = s_g(q_{g,i} - z_g). \quad (3)$$

Each group therefore contributes G four-bit codes – packed two per byte into a `qweight` tensor – plus one scale s_g and one zero-point z_g , stored in separate, much smaller `qzeros/scale` tensors. The ratio of weight-code bytes to metadata bytes is fixed by G and the chosen numeric format for s_g, z_g , independent of anything a downstream compressor can influence. This structural split – a small, per-group side channel carrying magnitude/shape information, and a large, per-weight residual channel carrying only fine placement within the group – is the basis for the mechanism identified in Section VI-B, and its consequences are visible directly in the per-tensor measurements reported there.

C. Zstandard Compression and the Entropy Floor

Zstandard combines LZ77-style sequence matching with entropy coding (Huffman coding for literals, FSE/tANS for sequence symbols) [4], [6]. For a memoryless (order-0) source with per-symbol entropy H , Shannon’s source coding theorem establishes that no lossless code can achieve an average rate below H bits/symbol, and that this bound is asymptotically achievable [8]. Zstd’s Huffman and FSE stages are, with respect to a tile’s marginal symbol statistics, order-0 entropy coders in this sense; only its LZ77 stage can do better, by exploiting short-range repeated substrings rather than marginal frequency alone. Consequently, a directly measured symbol entropy is a hard lower bound on what *any* entropy-coding stage – Zstd’s FSE, a from-scratch Huffman coder, or a future custom hardware entropy engine – can achieve on that data; only genuine repetition, exploited via LZ-matching or dictionary reference, can do better, and only to the extent real repeated substrings exist. This is the standard this paper’s measurements are held to, and the reason a measured near-maximal entropy (Section V-D) is a structural, not implementation-specific, finding.

The initial hypothesis motivating this architecture was that applying dictionary-based Zstd to fixed-size INT4 tiles would yield significant additional compression on top of quantization, meaningfully easing SRAM pressure. Testing that hypothesis directly is the subject of this paper.

This paper’s subject – lossless compression of a model’s *own* weights, for storage and on-chip movement – is distinct from a separate line of work that uses a language model *as* a general-purpose compressor for arbitrary text, exploiting the well-known connection between next-token prediction and entropy coding [14]. FineZip [14], for instance, uses an LLM’s predictive distribution to arithmetic-code arbitrary text far more effectively than conventional compressors, at the cost of running full LLM inference per compressed token – a fundamentally different resource trade-off (compute-for-ratio,

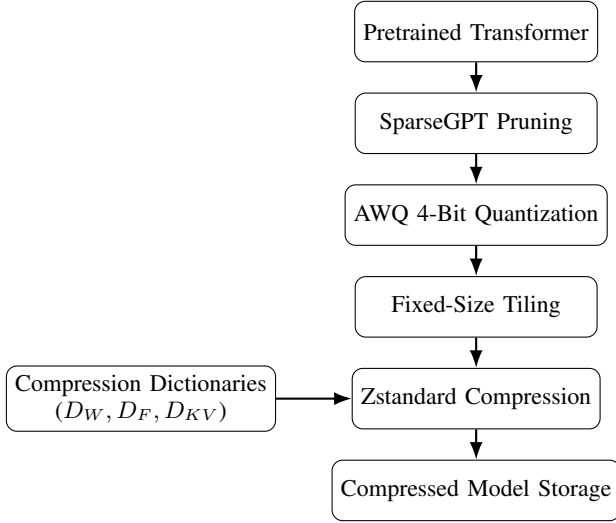


Fig. 1. Evaluated pipeline flow with compression dictionaries. The bottleneck reported in this paper is localized to the Zstandard Compression stage: everything upstream of it (pruning, quantization, tiling) behaves as designed.

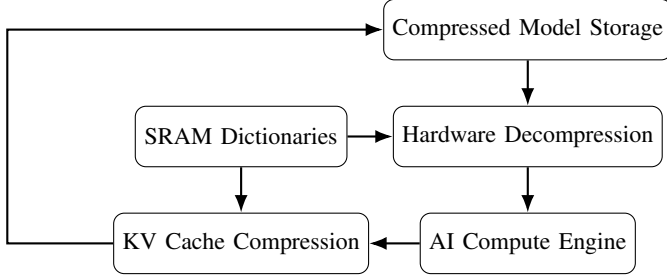


Fig. 2. Evaluated runtime architecture with SRAM dictionaries.

on someone else’s data) than this paper’s problem (storage-for-latency, on the model’s own parameters). The two problems share an entropy-coding vocabulary but are not interchangeable, and a result for one should not be assumed to transfer to the other.

III. EVALUATED COMPRESSION ARCHITECTURE

To test the feasibility of entropy coding at the edge, we designed a memory-centric architecture treating compression as a first-class hardware citizen (Fig. 1 and Fig. 2).

The architecture relies on tile-wise compression with role-specific dictionaries:

$$D = \{D_W, D_F, D_{KV}\} \quad (4)$$

where D_W is for weights, D_F is for feed-forward network tensors, and D_{KV} is applied dynamically to KV-cache data generated during autoregressive decoding. All three dictionaries occupy SRAM continuously, an overhead that must be weighed against whatever compression they enable (Section VIII).

IV. HARDWARE IMPLEMENTATION

To validate the architectural mechanics before assessing statistical viability, a complete hardware decompression dat-

apath and a hardware KV-cache compression datapath were implemented in SystemVerilog RTL and verified against independent golden C reference models. This section gives an overview of the datapath; Section VII reports what verification was actually performed and what it did and did not establish.

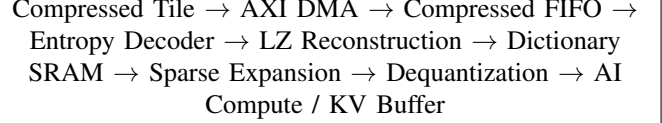


Fig. 3. Implemented streaming hardware datapath (read side).

The read-side datapath handles AXI DMA fetches, FSE/tANS entropy decoding, LZ77 reconstruction, dictionary fetches via address mapping $A_D = \text{offset} - T$, sparse expansion, and AWQ/INT8 dequantization. The write-side KV-cache compressor operates in reverse as a batch encoder: because tANS/FSE encoding requires the encode state resulting from later symbols before it can determine the bits to emit for an earlier one, the encoder must buffer a full tile before entropy encoding can begin, which is not expected to break the KV path’s real-time budget since the downstream attention operation requires the same full tile to already be written.

V. EXPERIMENTAL METHODOLOGY AND MEASURED COMPRESSION BOUNDS

The experimental evaluation aimed to quantify the compression bounds at each architectural stage using the Qwen3-8B checkpoint [1]. AWQ-INT4 quantization and SparseGPT (50%) pruning have each been applied and measured directly, in isolation; their combination has not yet been measured and remains projected, as noted in Section II-A.

A. Pipeline-Level Compression

TABLE I
MODEL COMPRESSION PIPELINE (8B-PARAMETER REFERENCE MODEL)

Configuration	Size (GB)	Ratio	Status
FP16 baseline	16.0	1.00×	Reference
AWQ-INT4 (no sparsity)	≈ 4.4	3.63×	Measured
AWQ-INT4 + Zstd (tile+dict)	≈ 3.98	≈ 4.02×	Measured
SparseGPT (50%)	8.5	1.88×	Measured^a
SparseGPT + AWQ-INT4	2.4	6.66×	Projected
SparseGPT + AWQ + Zstd	2.16	7.41×	Projected

^aMeasured in isolation (pruned, unquantized checkpoint); the two combined rows require pruning followed by quantization of the resulting sparse weights on a single checkpoint, not yet performed.

The AWQ-INT4+Zstd row is derived from the measured tile+dictionary ratio in Table IV applied to the measured AWQ-INT4 size. Both single-stage ratios (3.63× for AWQ-INT4, 1.88× for SparseGPT) were measured directly and land within rounding of their originally projected values – $16.0/3.63 = 4.41$ GB and $16.0/1.88 = 8.51$ GB, both matching the table to two significant figures – which is a useful consistency check but does not by itself validate the two combined-pipeline rows, which remain projections.

B. Tile Size

TABLE II
TILE SIZE (MEASURED ON AWQ-INT4)

Tile Size T	Ratio $R(T)$	Status
4096 bytes	$1.094\times / 1.107\times$ (dict)	Measured

Only one tile size has been measured to date. A sweep matching the RTL’s 16-byte verification granularity through larger software-side tile sizes is identified as future work (Section IX); given the entropy-floor finding below, however, no tile size is expected to change the qualitative conclusion, since tiling can only ever reduce, never increase, the compression a compressor achieves relative to an unbounded window (Table IV).

C. Dictionary Organization

TABLE III
DICTIONARY ORGANIZATION MEASUREMENTS

Configuration	Attention	FFN	Status
No Dict. (tile-wise)	$1.074\times$	$1.099\times$	Measured
Per-Tensor (held-out)	$1.088\times$	$1.112\times$	Measured

A held-out-validated per-tensor dictionary improves the tile-wise ratio by +1.3% relative (attention) and +1.2% relative (FFN). This is a real, measured effect, not zero – but it is small enough that the SRAM cost of maintaining a dictionary must be justified against a single-digit-percent gain, not the much larger gain the architecture was originally designed around.

D. The Entropy Bottleneck

TABLE IV
MEASURED LOSSLESS COMPRESSION, QWEN3-8B AWQ-INT4 WEIGHTS
(342.3 MB ACROSS 40 TENSORS)

Metric	Value
Byte entropy (floor: 8.0 bits/byte)	7.281
Nibble entropy (floor: 4.0 bits/sym)	3.654
zlib –9, whole tensor	$1.109\times$
zstd, whole tensor (unbounded window)	$1.139\times$
zstd, 4096 B tiles (hardware-achievable)	$1.094\times$
zstd, tiles + dictionary	$1.107\times$

The central empirical finding is the measured nibble entropy: **3.654 of a possible 4.0 bits per 4-bit symbol**, i.e. 91.4% of the theoretical maximum. By the bound established in Section II-C, no order-0 entropy coder – Zstd’s FSE included – can recover much more than the ~ 9 –14% observed here on this data. The redundancy reduction has, to first order, already been completed by the AWQ step. Section VI identifies the specific mechanism by which this happens, rather than treating it as an unexplained empirical fact.

VI. MECHANISM: HOW QUANTIZATION CONSUMES COMPRESSIBILITY

The finding in Section V-D raises an obvious follow-up question: is quantization merely *neutral* with respect to compressibility – consuming redundancy that a compressor would have found anyway, with no further consequence – or does it actively work against the compressor in some identifiable way? The per-tensor measurements available from this study support the latter, more specific claim.

A. Accuracy-Optimized Quantization Tends Toward High Entropy

AWQ’s optimization objective minimizes activation-weighted reconstruction error; it contains no explicit term rewarding or penalizing the entropy of the resulting integer codes. A highly skewed, easily-compressible code distribution is therefore not something AWQ avoids by design – it would simply have to arise as an accidental byproduct of accuracy optimization. Empirically, it does not arise: because each group’s scale s_g is fit to that group’s own value range (Section II-B), the re-normalized, per-group weight distribution that gets mapped onto the 16 available codes is, group by group, a locally well-matched fit rather than a badly-clipped or badly-underutilized one. A quantizer whose codes were strongly skewed within a well-fit group would, in effect, be leaving reconstruction accuracy on the table by using its 4-bit budget unevenly. The measured near-maximal entropy is consistent with AWQ’s per-group scaling doing its accuracy-optimization job efficiently, not with it failing at some other, unstated objective.

B. Structural Relocation: Scale/Zero-Point vs. Weight Codes

The stronger and more specific finding is visible when `qweight` (the bulk 4-bit codes) and `qzeros/scale` tensors (the per-group metadata from Section II-B) are examined separately rather than pooled together, as Table IV does. Table V reports both tensor types for every projection matrix in one transformer layer for which a complete matched pair was measured.

TABLE V
COMPRESSION BY QUANTIZATION ARTIFACT TYPE, ONE TRANSFORMER LAYER

Projection	<code>qweight</code> H / Ratio	<code>qzeros</code> H / Ratio	Size Ratio
down_proj	7.34 / $1.076\times$	3.71 / $2.046\times$	126:1
gate_proj	7.32 / $1.081\times$	4.03 / $1.936\times$	126:1
up_proj	7.33 / $1.078\times$	3.59 / $2.090\times$	126:1
o_proj	7.38 / $1.072\times$	3.22 / $2.258\times$	120:1
q_proj	7.25 / $1.091\times$	4.88 / $1.569\times$	120:1
Mean	7.32 / $1.080\times$	3.89 / $1.980\times$	≈ 120 –126:1

H in bits/byte; ratio is tile+dictionary Zstd. Size ratio is `qweight` bytes : `qzeros` bytes for that projection.

The pattern is consistent across every measured pair: the small scale/zero-point tensors compress at **1.98 $\times$** on average – nearly double the **1.08 $\times$** achieved on the weight codes they were split off from – while representing under 1% of

the combined bytes (a size ratio of $\approx 120\text{--}126:1$, consistent with a uniform group size on the order of $G \approx 128$ across all projections, matching common AWQ configurations). Because qzeros tensors are this small a fraction of total bytes, the pooled per-role averages already reported in Table IV and Table III are effectively determined by qweight alone; separating the two tensor types changes the reported ratios negligibly at the aggregate level but reveals *where* the compressible structure actually is.

This is the paper’s central mechanistic claim: AWQ’s group-wise affine factorization does not simply fail to leave redundancy behind for Zstd. It *relocates* the redundancy that does exist – the smooth, structured, per-group magnitude information that in an unquantized representation would live in something like a float exponent field (Section VI-D) – into a side channel explicitly sized to be as small as possible, by design, since minimizing metadata overhead is itself part of what makes a quantization scheme practical. The residual 4-bit codes, which constitute over 99% of the storage, are what remains after that relocation, and what remains is close to entropy-maximal by construction. A compression architecture built to operate on qweight tensors is therefore, in a precise sense, targeting exactly the part of the representation that quantization was designed to strip of exploitable structure, while the part that does retain structure is too small to matter for overall footprint even if compressed perfectly.

C. Nibble-Packing Independence Check

A secondary, corroborating measurement: two 4-bit codes are packed into each qweight byte. If the mean nibble entropy (3.654 bits, Table IV) is representative and the two packed nibbles were statistically independent, the resulting byte entropy would be predicted as $2 \times 3.654 = 7.308$ bits. The directly measured byte entropy is 7.281 bits – within 0.4% of this prediction. This indicates the two nibbles packed into each byte carry no additional exploitable cross-correlation beyond their individual near-maximal entropy: nibble-packing itself does not introduce or destroy structure, it simply concatenates two already-close-to-random symbols.

D. Comparison Against Unquantized FP16

An earlier design-time projection for this architecture (an earlier revision of Table II, since withdrawn) estimated a $1.25\times$ ($\sim 25\%$) lossless compression margin at larger tile sizes; that projection was for the *same* AWQ-INT4 representation at untested tile sizes, not for FP16 weights, and it did not survive measurement even approximately at the one tile size actually tested ($1.094\text{--}1.107\times$).

A separate and more decisive question is whether *avoiding* quantization – compressing dense FP16 weights directly, using the identical tile-wise methodology – would fare better. Direct measurement on the dense Qwen3-8B checkpoint shows that it does: Zstd achieves $1.254\times$ on FP16 weights, against $1.107\times$ on AWQ-INT4 (Table VI). This is a real, measured effect, not a hypothesis, and it sharpens this paper’s central claim from “quantization leaves little for Zstd” to something

stronger: *quantization measurably reduces the fraction of the representation that Zstd can subsequently recover.*

TABLE VI
ZSTD YIELD: FP16 VS. AWQ-INT4, SAME METHODOLOGY

Representation	Zstd Ratio	Size Reduction	Status
FP16 (dense)	$1.254\times$	20.3%	Measured
AWQ-INT4	$1.107\times$	9.7%	Measured

Expressed as the fraction of remaining size Zstd removes, FP16 gives up 20.3% while AWQ-INT4 gives up only 9.7% – Zstd recovers roughly *half* the relative benefit on the quantized representation that it recovers on the unquantized one, measured under the same tile size and compressor settings. This is consistent with, and now confirms rather than merely predicts, the mechanism argued on structural grounds in Section VI-B: an IEEE-754 half-precision value is 1 sign bit, 5 exponent bits, and 10 mantissa bits, and only the exponent field – at most half the bytes – clusters around a tensor’s typical magnitude in a way a compressor can exploit; the mantissa is close to uniformly distributed at the bit level regardless of how skewed the value distribution looks. AWQ’s per-group scaling (Section II-B) does something structurally similar but more extreme: it factors out exactly this kind of magnitude information into a side channel under 1% the size of FP16’s exponent field, leaving a residual with correspondingly less to exploit. FP16 is the less extreme case of the same phenomenon Section VI-B measures directly for AWQ-INT4, and the real measurement confirms that ordering.

One caveat on magnitude: a controlled synthetic check at matched weight scale ($w \sim \mathcal{N}(0, 0.02)$, i.i.d. per value) predicted a substantially lower FP16 ratio (mantissa byte at 7.998 of 8 bits, sign+exponent byte at 5.462 of 8 bits, implying $\approx 1.08\times$) than the $1.254\times$ actually measured. Real trained weights evidently carry more Zstd-exploitable structure than an i.i.d. Gaussian model captures – plausibly cross-weight correlations, non-Gaussian tails, or exact repeated substrings from architectural regularities (e.g. tied parameters, repeated block structure) that a per-value-independent synthetic model cannot produce. The byte-position mechanism correctly predicts the *direction* of the FP16-vs-AWQ-INT4 comparison; it understates FP16’s absolute compressibility, and this gap is itself informative: it indicates that whatever extra structure real FP16 weights carry beyond simple magnitude clustering is exactly the kind of structure AWQ’s quantization removes, since AWQ-INT4’s measured ratio ($1.107\times$) sits much closer to the simple synthetic prediction ($\approx 1.08\times$) than FP16’s does.

Accounting for total footprint rather than the Zstd stage alone: FP16+Zstd reaches $16.0/1.254 \approx 12.76$ GB, while AWQ-INT4+Zstd reaches ≈ 3.98 GB (Table I) – AWQ-INT4+Zstd is still $\approx 3.2\times$ smaller in absolute terms. Quantization’s own $3.63\times$ packing gain vastly outweighs the smaller, but now measured and real, opportunity cost it imposes on the subsequent Zstd stage.

E. Comparison Against the Literature: Is $\sim 10\%$ Actually Bad?

Generic byte-oriented Zstd, the compressor used throughout this paper, is not the strongest published lossless technique for FP16 weights specifically, and the $1.254\times$ figure above should not be read as a ceiling. Huff-LLM [9] reports $1.16\text{--}1.47\times$ across model families using a field-decomposed Huffman coder that splits each FP16 value into sign, exponent, and two mantissa halves and Huffman-codes each independently with its own codebook – and specifically reports $1.46\times$ (31.5% reduction) for Qwen-2.5-7B, the closest published analog to the Qwen3-8B checkpoint measured here. Three further independent works report similar ranges via a related exponent-focused strategy: ZipNN [10] reports 17–33% on FP16/BF16 models; NeuZip [11] reports 33% on BF16 (at the cost of a 33% inference slowdown from decompression); and DFloat11 [12] reports a consistent $\approx 30\%$ reduction across Llama-3.1, Qwen-2.5, and Gemma-3 by Huffman-coding only the BFloat16 exponent field, which it measures directly at ≈ 2.6 of 8 possible bits – a 32.5% occupancy figure remarkably close to this paper’s own 2.541-of-5-bit (50.8%) measurement for the FP16 exponent (Section V-D), consistent with BF16’s wider 8-bit exponent field carrying proportionally more redundancy than FP16’s narrower 5-bit one, which is also why DFloat11 and ZipNN both report their higher ratios specifically on BF16 rather than FP16. Four independent groups therefore converge on the same underlying mechanism argued structurally in Section VI-D above: FP16/BF16 compressibility is concentrated almost entirely in the exponent field, and a compressor that isolates that field with its own codebook – rather than compressing raw, field-misaligned bytes – recovers meaningfully more of it. This is independent triangulation of this paper’s own byte-position mechanism finding, not a competing result, and it means the true achievable FP16 ceiling for this checkpoint’s model family is closer to Huff-LLM’s 31.5% than to this paper’s own 20.3%.

This raises the question directly: if the literature treats $\sim 30\%$ lossless compression of FP16 weights as a good, publication-worthy result, is this paper’s $\sim 10\%$ marginal result on AWQ-INT4 actually a negative one, or is it simply being held to an unfairly higher bar? The two numbers answer different questions relative to different baselines, and conflating them is the error to avoid, not the comparison itself. Huff-LLM asks: starting from full FP16 weights, with no lossy transformation applied at all, how much can a purpose-built lossless coder save? Its $\sim 30\%$ is valuable precisely because it comes at zero accuracy cost, which matters in settings – regulatory compliance, safety-critical reproducibility – where any lossy transformation, however small its measured accuracy impact, is unacceptable by policy rather than by benchmark score. This paper asks a narrower, sequential question: starting from a checkpoint that has *already* been quantized, how much additional, marginal value does a further lossless stage add on top? These are not comparable performance claims for the same task; they are answers to two different design questions,

and a design that has already committed to accepting AWQ’s lossy transformation has, by that choice, already captured most of the available compressibility – there is materially less left over for any subsequent lossless stage, however well designed, to find. The $\sim 10\%$ measured here is not 30% done badly; it is the correct, much smaller number for a different, harder-to-improve starting point.

The architectural implication sharpens rather than softens under this comparison. Even Huff-LLM’s best published ratio for a Qwen-family model ($1.46\times$) applied to this paper’s 16 GB FP16 baseline gives 10.96 GB with *no* lossy transformation at all – still $2.49\times$ larger than AWQ-INT4 quantization achieves *by itself*, before any lossless stage is applied (Table I). A field-aware entropy coder is a genuinely good idea for a system that has ruled out lossy compression on policy grounds; it is not a substitute for quantization when a policy has not ruled that out, and it is not, on the evidence gathered here, a substantial enough addition *on top of* quantization to justify dedicated hardware for that combination specifically. There is a further, structural reason to expect Huff-LLM’s specific technique would not transfer to the AWQ-INT4 setting even if implemented: its gain comes from decomposing a value into internal sub-fields (exponent vs. mantissa) with different compressibility and coding each separately. A 4-bit AWQ code has no such internal sub-field structure to decompose – the whole 4 bits *is* the residual left over after AWQ’s own group-wise scaling has already performed the analogous separation (Section VI-B), extracting the exponent-like magnitude information into the `qzeros`/scale side channel. Section VI-B’s finding and Huff-LLM’s finding are, in this sense, the same mechanism observed on either side of the same design choice: whichever stage performs the separation first – AWQ’s quantizer, or a field-aware entropy coder acting on unquantized weights – captures the available structure, and there is little left for whichever stage comes second.

The correct reading of this section’s finding is therefore not that quantization was a mistake, but that quantization and lossless entropy coding are not additive, independent sources of compression: they compete for the same underlying redundancy, quantization claims most of it first by design, and any architecture budgeting hardware area for the entropy-coding stage should size that budget to the $\sim 10\%$ margin actually available after quantization, not to a margin estimated from the unquantized representation.

VII. RTL VERIFICATION STATUS

The statistical finding above is a property of the data, not of implementation quality; establishing that required verifying the hardware rigorously enough to trust it as a measurement instrument, which this section documents.

A. Decompression Pipeline (`zstd_decomp_top`)

The read-side RTL was checked with a byte-exact scoreboard against an independent golden C reference model implementing the same stage ordering (entropy decode $\rightarrow$ LZ77 $\rightarrow$ despars $\rightarrow$ dequant). For a single tile combining

six literal symbols and one six-byte in-tile LZ77 match, the entropy-decode and LZ-reconstruction stages produced output matching the golden model bit-exactly at every offset. Two adversarial checks establish this was not a vacuous pass: (i) an independently generated golden vector with a different symbol sequence, tile length, and match offset/length also matched bit-exactly, confirming the RTL decodes rather than reproduces a fixed answer; and (ii) a deliberate single-line mutation of the entropy decoder’s state-advance computation produced an immediate scoreboard mismatch, confirming the scoreboard can detect a real defect.

Three defects were found and fixed during this bring-up. A table-size parameter override (`STATE_BITS`) had been removed under the reasoning that the architectural default width was sufficient; this reintroduced a simulator-specific defect in which a large (4096-entry) packed-struct array is silently mis-written by the Icarus Verilog 12.0 toolchain, stalling the decoder indefinitely. A double-buffer handshake defect (`buf_swap_ack`) caused the tile scheduler to wait indefinitely for an acknowledgment pulse that, by construction of the surrounding FSM, had already necessarily elapsed. An elaboration-time defect causes `NUM_TILES=1` to fail to compile under this toolchain ($\log_2(1) = 0$ producing zero-width constructs at 24 sites); the testbench works around this by instantiating capacity for two tiles and running one.

Multi-tile operation with the genuine entropy stage, dictionary-crossing LZ matches (a specific timing defect has already been identified in the dictionary read path), and standards-compliant Zstd bitstream compliance (the current entropy stage uses a deliberately simplified power-of-2-frequency FSE table construction, not the general Zstandard construction) remain unverified.

B. KV-Cache Compressor (`kv_compress_top`)

A per-stage scoreboard checks the quantizer, LZ match finder, and entropy encoder independently against the golden model. The pipeline is not yet functionally verified: the quantizer stage’s output is offset by one beat relative to the golden model, confirmed as an RTL defect (reproduced identically under two different handshake disciplines) rather than a testbench artifact. Because every downstream stage depends on this stage’s output, the match finder and encoder have not been meaningfully exercised beyond this point.

One defect was found and fixed prior to isolating the above: a tile-length register in the LZ match finder (`tile_len_q`) was one bit too narrow, silently wrapping to zero and terminating the match finder after a single token, which had manifested as a full-pipeline timeout with no diagnostic information. A structural gap was also identified: the top-level module’s `start` and `tile_id` inputs are not connected to any control logic, so `busy/done` currently reflect only the AXI write master’s state, not tile-level completion.

Given the mechanism identified in Section VI, it would be a mistake to assume the KV-cache path shares the weight path’s entropy bottleneck by analogy: KV entries are runtime activations, not the output of an accuracy-optimizing quantizer

designed to spread information evenly across a fixed code budget. Whether real, exploitable redundancy survives in KV data is a genuinely open, unmeasured question (Section IX), and this paper takes no position on it either way.

VIII. DISCUSSION

The central finding of this feasibility study is that combining lossy weight quantization with lossless entropy coding yields sharply diminishing returns, and does so for an identifiable structural reason rather than an unexplained empirical accident. The hardware pipelines for Zstd decompression and KV-cache compression were designed, implemented in RTL, and functionally verified at the single-tile level with genuine rigor (Section VII); the bottleneck reported here is not attributable to an implementation shortcut.

An earlier design-time projection for this architecture anticipated a substantially larger lossless margin at larger tile sizes (Table II, discussion in Section VI-D); the measured margin at the one tile size actually tested is $1.09\text{--}1.11\times$, well short of that projection. Section VI’s mechanism explains why: AWQ’s group-wise scaling does not fail to produce compressible structure so much as it deliberately concentrates that structure into a side channel ($\approx 1.98\times$ compressible) too small ($< 1\%$ of bytes) to matter for overall footprint, leaving the dominant weight-code tensors ($\approx 1.08\times$) to carry over 99% of the storage at close to their entropy floor. Historical and contemporary Huffman-based approaches to network and LLM weight compression are, for the marginal contribution of any entropy-coding stage specifically, subject to the same Shannon bound established in Section II-C; Section VI-E compares this paper’s finding directly against three such published results and shows the apparent tension – $\sim 10\%$ here versus $\sim 30\%$ reported elsewhere – is not a contradiction but a difference in what baseline each number is measured against.

Given the substantial hardware complexity, SRAM dictionary capacity, control logic, and verification effort required to implement dynamic entropy decompression for model weights and KV caches, a measured $\sim 10\%$ margin carries a high burden of proof before further silicon investment is warranted. This burden is sharpened, not merely asserted, by Section VI-B’s finding that even a perfectly-compressed scale/zero-point channel could not move the aggregate ratio meaningfully, because that channel is too small to matter regardless of how well it compresses. This negative analysis suggests that edge inference accelerators’ memory-reduction budgets are better directed at sparsification and quantization scheme design – where the factor-of-several gains actually live – than at entropy-coding wrappers built to extract redundancy that a well-designed quantizer has already, by design, removed.

IX. LIMITATIONS AND FUTURE WORK

This paper’s negative conclusion is scoped specifically to lossless entropy coding of AWQ-INT4-quantized static weights, on the one checkpoint, tile size, and dictionary configuration measured. The following remain open and should not be assumed to share the same outcome:

- 1) **KV-cache compressibility is entirely unmeasured**, in software or hardware. As argued in Section VII-B, there is no a priori reason to expect it to share the weight path’s entropy bottleneck, since KV data does not pass through an accuracy-optimizing quantizer analogous to AWQ. This is the single highest-priority remaining experiment.
- 2) **SparseGPT has been applied and measured in isolation** ($1.88\times$, Table I), but combining it with AWQ-INT4 quantization on a single checkpoint has not; whether pruning’s exact zeros compress differently once co-located with AWQ’s non-zero residual codes is untested and could, in principle, partially offset this paper’s conclusion for a sparsified-and-quantized model.
- 3) **Throughput has not been characterized** for the genuine (non-passthrough) entropy-coded pipeline. The current single-FSE-stream decoder is architecturally bounded to approximately one symbol per cycle; this is a separate question from the compression-ratio finding and does not affect it either way. This is not a niche concern: ZipServ [13] identifies the same variable-length-code/SIMT-parallelism mismatch as a fundamental obstacle to GPU-side lossless LLM inference and addresses it by replacing entropy coding with a fixed-length bitmap format (Tensor-Core-Aware Triple Bitmap Encoding) that trades some compression ratio for constant-time, fully parallel decode. Whether an analogous fixed-length encoding is preferable to interleaved FSE streams for this paper’s RTL datapath specifically – trading a small amount of the already-modest $\sim 10\%$ ratio for materially simpler, higher-throughput hardware – is a concrete, testable extension of this future-work item rather than a settled question.
- 4) **Generalization beyond Qwen3-8B and AWQ** is untested. Whether the near-maximal-entropy result holds for other activation-aware quantizers, other model families, or other group sizes is an open empirical question this single-model study cannot answer.

X. CONCLUSION

This paper reports a rigorously verified negative result: for AWQ-INT4-quantized Qwen3-8B weights, tile-wise Zstandard compression contributes only a $1.09\text{--}1.11\times$ margin beyond quantization, bounded by a measured nibble entropy of 3.65 of 4.0 bits. Beyond quantifying this bottleneck, this paper identifies its mechanism: AWQ’s group-wise affine quantization splits each weight into a compressible scale/zero-point component and a residual 4-bit code, and – as measured directly across five matched tensor pairs – concentrates essentially all recoverable structure ($\approx 1.98\times$) into the former, which is under 1% of stored bytes, leaving the latter, over 99% of storage, close to its entropy floor ($\approx 1.08\times$). This is not an incidental byproduct of quantization; it is close to what an accuracy-optimized, per-group quantizer would be expected to do to a smooth, structured source under a fixed bit budget, and a direct measurement against the unquantized

dense checkpoint confirms this: Zstd recovers a 20.3% size reduction on FP16 weights but only 9.7% on AWQ-INT4, roughly half. Quantization was not, however, a mistake – its own $3.63\times$ packing gain (also now directly measured, as is SparseGPT’s $1.88\times$ in isolation) leaves the combined AWQ+Zstd pipeline $\approx 3.2\times$ smaller than FP16+Zstd despite this opportunity cost. The hardware built to test this hypothesis was implemented and verified with real rigor – including adversarial mutation testing and two independently confirmed defects, described in Section VII – so the negative result reported here reflects a property of the data, not a shortfall in the engineering used to measure it. Given this, we conclude that further hardware investment in general-purpose entropy coding for post-quantization edge LLM weights is not well justified by current evidence, and that the KV-cache path – for which no such measurement yet exists – is the appropriate place to direct the next experiment before drawing the same conclusion architecture-wide.

REFERENCES

- [1] Qwen Team, “Qwen3 Technical Report,” arXiv:2505.09388, 2025.
- [2] E. Frantar and D.-A. Alistarh, “SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot,” in *Proc. 40th Int. Conf. Machine Learning (ICML)*, vol. 202, pp. 10323–10337, 2023.
- [3] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration,” in *Proc. MLSys*, 2024.
- [4] Y. Collet and M. Kucherawy, “Zstandard Compression and the ‘application/zstd’ Media Type,” RFC 8878, Internet Engineering Task Force, Feb. 2021.
- [5] S. Karandikar, A. N. Udiipi, J. Choi, J. Whangbo, J. Zhao, S. Kanev, E. Lim, J. Alakuijala, V. Madduri, Y. S. Shao, B. Nikolic, K. Asanovic, and P. Ranganathan, “CDPU: Co-designing Compression and Decompression Processing Units for Hyperscale Systems,” in *Proc. 50th Int. Symp. Computer Architecture (ISCA)*, 2023, doi: 10.1145/3579371.3589074.
- [6] Y. Collet, “Zstandard Reference Manual,” Meta Platforms, Inc., 2026.
- [7] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *Int. Conf. Learning Representations (ICLR)*, 2016.
- [8] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423, 623–656, 1948.
- [9] P. Yubeaton, T. Mahmoud, S. Naga, P. Taheri, T. Xia, A. George, Y. Khalil, S. Q. Zhang, S. Joshi, C. Hegde, and S. Garg, “Huff-LLM: End-to-End Lossless Compression for Efficient LLM Inference,” arXiv:2502.00922, 2025.
- [10] M. Hershcovitch, A. Wood, L. Choshen, G. Girmonsky, R. Leibovitz, I. Ennmouri, M. Malka, P. Chin, S. Sundararaman, and D. Harnik, “ZipNN: Lossless Compression for AI Models,” arXiv:2411.05239, 2024.
- [11] Y. Hao, Y. Cao, and L. Mou, “NeuZip: Memory-Efficient Training and Inference with Dynamic Compression of Neural Networks,” arXiv:2410.20650, 2024.
- [12] T. Zhang, M. Hariri, S. Zhong, V. Chaudhary, Y. Sui, X. Hu, and A. Shrivastava, “70% Size, 100% Accuracy: Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float,” arXiv:2504.11651, 2025.
- [13] R. Fan, X. Yu, X. Pan, Z. Li, W. Luo, Q. Wang, W. Wang, and X. Chu, “ZipServ: Fast and Memory-Efficient LLM Inference with Hardware-Aware Lossless Compression,” in *Proc. 31st ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, vol. 2, 2026, pp. 2264–2280.
- [14] F. Mittu, Y. Bu, A. Gupta, A. Devireddy, A. E. Ozdarendeli, A. Singh, and G. Anumanchipalli, “FineZip: Pushing the Limits of Large Language Models for Practical Lossless Text Compression,” arXiv:2409.17141, 2024.